



II SEMESTER 2025-2026

Assignment

Course No.: CS F422

Course Title: Parallel Computing

Deadline: As per Canvas

Maximum Marks: 40M (20%)

Note:

- Maximum of two students per group.
 - Demo & viva will be scheduled after submission.
-

P1. Consider the following dataset that contains reviews for Movies & TV on Amazon website. Please implement the following.

Dataset: [review dataset](#) Dataset description: [decsription](#)

- (a) Implement a CUDA program `cuda_toprated.cu` for listing the names of top 10 rated movies. The rating of a movie is defined as the average of all individual user ratings for that movie. Each CUDA thread should process one review record, and reduction should be used to aggregate ratings per movie across all reviews
- (b) Also implement `cuda_toprated_optimized.cu` that applies all possible optimizations given in book R6 to (a). Optimizations to consider include: Memory coalescing and aligned global memory accesses, shared memory usage to reduce global memory bandwidth, warp-level primitives and reduction techniques, minimizing thread divergence, use of CUDA streams for overlapping computation and data transfer
- (c) Compare the performance of the two in (a) and (b). Your comparison must include: Execution time measured using CUDA events (`cudaEventRecord`), speedup ratio (optimized vs baseline), memory throughput and occupancy analysis using Nsight or nvprof, discussion of which optimizations contributed most to performance gain. Document the comparison results in the Design Document.
- (d) Implement a CUDA program `cuda_reviewanalysis.cu` that analyzes the reviews and labels them as positive (if sentiment score > 0) and negative (if sentiment score < 0). Make use of [lexicon sentiment scores](#) and calculate sentiment score as lexicon frequency * lexicon score. Lexicon can be combination of more than one word. Each CUDA thread should process one review. The lexicon should be loaded into constant or shared memory for fast access.
- (e) Implement a sequential C/C++ program `c_elaborate.c` to identify reviewers who have written elaborate reviews. Elaborate reviewers are defined as `len(review text) >= 50` words and at least 5 such reviews. The program should output the reviewer IDs and their review count and average review length.
- (f) Apply OpenMP extensions to parallelize this program on GPUs and saved as `c_elaborate_openmp.c`. Your implementation must: Use `#pragma omp parallel for` to parallelize the review scanning loop, handle race conditions correctly using atomic operations or reduction clauses, experiment with different thread counts (2, 4, 8, 16) and report the effect on runtime, ensure output is identical to the sequential version in (e)



Birla Institute of Technology & Science, Pilani

Pilani Campus

- (g) Compare the performance of the two in (e) and (f). Your comparison must include: Execution time for both programs on the full dataset, Speedup achieved at different OpenMP thread counts, Analysis of Amdahl's Law limitations observed in the parallelized version. Document the comparison results in the Design Document.

Deliverables:

- Well commented source code files for (a), (b), (d), (e), (f)
- Design Document (.pdf) containing design aspects and optimizations. Also must contain answers for (c), and (g).

[40M]

--&--